UNIVERSIDAD TECNOLÓGICA NACIONAL · FACULTAD REGIONAL CÓRDOBA

INGENIERÍA Y CALIDAD DE SOFTWARE (ISW)
Resumen Institucional Consolidado — Segundo Parcial

Departamento de Ingeniería en Sistemas de Información · Cátedra de Ingeniería y Calidad de Software · Ciclo Lectivo 2026

Propósito del Documento: Compendio académico y técnico exhaustivo de la totalidad de los contenidos teóricos y prácticos evaluados en la segunda instancia parcial de la asignatura (Notebook Oficial ISW — UTN FRC), estructurado y nominado bajo las normas de Gestión de Configuración de Software (SCM) del repositorio oficial de cátedra **ISW_4K1_G1_2026**.

Ficha de Identificación del Ítem de Configuración (Matriz de ICs)

Atributo de Configuración	Especificación Institucional
Repositorio SCM	`ISW_4K1_G1_2026` (Curso: 4K1 · Comisión/Grupo: Grupo 1 · Año: 2026)
Tipo de Ítem / Clasificación	`produccion_propia` — Ítem de Configuración: **Resumen Consolidado de Cátedra**
Regla de Nombrado Aplicada	`resumen-u<numero_unidad>-<tema>-<ApellidoAutor_NombreAutor>.<ext>`
Nombre Físico del Archivo (Grupal / Individual)	`resumen-u04-segundoparcial-grupo1.pdf` (Equivalente individual: `resumen-u04-segundoparcial-Duarte_Cayetano.pdf`)
Ubicación Física en el Repositorio	`ISW_4K1_G1_2026/produccion_propia/resumenes/`
Línea Base (Git Tag Anotado)	`lb-parcial-02` (Hito: Evaluación Formal Segundo Parcial — Unidades 02, 03 y 04)
Autor / Responsable de Elaboración	Cayetano Amaro Duarte (`Duarte_Cayetano` / `grupo1`)
Estado de Configuración	Versión Definitiva Consolidada (Verificada por Peer Review, sin marcas de borrador)

Alcance Temático Evaluado en el Segundo Parcial

Unidad	Título Oficial según Programa 2026	Ejes Conceptuales y Prácticos Incluidos
Unidad 02	Gestión Lean-Ágil de Productos de Software (Parte II)	Filosofía Lean (7 principios y 7+1 desperdicios Muda), Método Kanban (Principios, 6 Prácticas, WIP Limits, Políticas Explícitas en 5 ejes, 4 Clases de Servicio, 7 Cadencias), Métricas en los 3 Enfoques (Tradicional EVM, Ágil y Lean/Ley de Little/CFD), Design Thinking, Lean UX y Retrospectivas (5 etapas).
Unidad 03	Gestión del Software como Producto (Parte II)	Prácticas Continuas (Continuous Integration CI, Continuous Delivery CDE, Continuous Deployment CD) y Estrategias de Despliegue (Big Bang, Rolling Update, Blue-Green, Canary Release, A/B Testing, Feature Toggles, Shadow Deployment).
Unidad 04	Aseguramiento de Calidad de Proceso y de Producto (Completa)	Conceptos de Calidad (Crosby, Juran, Deming, ISO, Pressman, Costos de Calidad), SQA y Modelos (CMMI Niveles y Área PPQA: SG1/SG2, ISO 9001/12207/15504 SPICE/25010 SQuaRE), Revisiones Técnicas (Fagan, Walkthrough, Pares), Auditorías de Software y Testing de Software (Niveles, Tipos, Caja Blanca/McCabe, Caja Negra: Partición de Equivalencia y Valores Límite, Agile Testing Q1-Q4, Pirámide de Cohn, TDD, BDD/ATDD).
Práctico 2P	Resolución de Casos de Prueba y Tablero Kanban	Algoritmo de Partición de Equivalencia y Valores Límite sin enmascaramiento de defectos (Template de Caso de Prueba) y diagnóstico/resolución de cuellos de botella en Tablero Kanban.

Fuentes Oficiales Trazables (Notebook ISW): `Temas segundo Parcial`, `Compiladao parcial segundo parcial`, `10 LEAN KANBAN.pdf`, `11 Métricas en los Diferentes Enfoques.pdf`, `12 PPQA_Intro.pdf`, `13 Revisiones Técnicas.pdf`, `14 Auditorías de Sw.pdf`, `15 Retrospectivas Intro Práctico.pdf`, `09 Testing de Software.pdf`, `09 Testing de Software Estrategias Caja Negra.pdf`, `09 Agile Testing.pdf`, `TDD.pdf` y exámenes `parcial2-2026-4k4.pdf` / `parcial2-2025-4k1.pdf`.

1. Unidad 02 (Parte II): Filosofía Lean, Método Kanban, Métricas y Retrospectivas

Fuentes de cátedra (Notebook ISW): `10 LEAN KANBAN.pdf`, `Essential-Kanban-Condensed-Spanish.pdf`, `The-Official-Kanban-Guide_Spanish_A4.pdf`, `11 Métricas en los Diferentes Enfoques.pdf`, `15 Retrospectivas Intro Práctico.pdf`.

1.1 Filosofía Lean Aplicada al Desarrollo de Software (Poppendieck)

Originada en el Sistema de Producción Toyota (TPS, Taiichi Ohno) bajo las premisas de eliminar el desperdicio (*Muda*), producir *Just-In-Time* (producir solo lo necesario, en el momento justo y en la cantidad necesaria; analizar cuando se necesite, no antes) y construir calidad en el origen (*Jidoka*), Mary y Tom Poppendieck adaptaron la filosofía Lean al software mediante **7 Principios Fundamentales**:

1. **Eliminar los desperdicios (*Eliminate Waste / Muda*):** Suprimir toda actividad, documento o código que consuma tiempo y recursos sin aportar valor real para el cliente.
2. **Amplificar el aprendizaje (*Amplify Learning*):** Tratar el desarrollo como un proceso de descubrimiento continuo mediante iteraciones cortas, feedback temprano e integración frecuente.
3. **Decidir lo más tarde posible (*Decide as Late as Possible*):** Postergar las decisiones arquitectónicas o de diseño irreversibles hasta el **"último momento responsable"**, manteniendo abiertas las opciones mientras se reúne más información real y reduciendo el costo de cambios.
4. **Entregar lo más rápido posible (*Deliver as Fast as Possible*):** Minimizar el tiempo de ciclo desde el pedido hasta la puesta en producción para validar valor real y evitar que el contexto cambie durante esperas largas.
5. **Empoderar / Dar poder al equipo (*Empower the Team*):** Confiar en las personas que hacen el trabajo técnico para tomar decisiones locales, fomentando equipos autoorganizados y liderazgo facilitador.
6. **Construir con integridad (*Build Integrity In*):** Lograr tanto **integridad percibida** (el sistema es coherente, útil y fácil de usar para el cliente) como **integridad conceptual** (arquitectura limpia, código refactorizado, TDD y pruebas automatizadas).
7. **Ver el todo (*See the Whole*):** Optimizar el flujo de valor completo de punta a punta (*end-to-end*) evitando las suboptimizaciones locales (donde un área acelera su parte pero satura de trabajo en espera al área siguiente).

#	Desperdicio en Manufactura (TPS)	Equivalente en Desarrollo de Software	Impacto Técnico y Ejemplos de Cátedra
1	Sobreproducción	**Funcionalidades Extra (*Extra Features / Gold Plating*)**	Codificar funciones "por las dudas" que el usuario no necesita (recordar que el 45% de las funciones nunca se usa); suma complejidad, bugs y costo de mantenimiento.
2	Inventario acumulado	**Trabajo Parcialmente Hecho (*Partially Done Work*)**	Requerimientos especificados sin codificar, código sin integrar, ramas sin fusionar o funciones sin testear ni desplegar; inmoviliza inversión sin dar valor.
3	Procesamiento extra	**Procesos Extra / Reaprendizaje**	Burocracia documental que nadie lee, aprobaciones redundantes o volver a relevar lo que ya se sabía por no compartir conocimiento.
4	Transporte	**Transferencias / Pasamanos (*Handoffs*)**	Pasar artefactos de mano en mano entre áreas aisladas (Analista → Diseñador → Programador → Tester); degrada el conocimiento tácito en cada traspaso.
5	Movimiento innecesario	**Cambio de Tarea (*Task / Context Switching*)**	Asignar a un desarrollador a múltiples proyectos o tareas a la vez; destruye entre el 20% y 40% de su capacidad productiva por el costo cognitivo de cambio de contexto.

#	Desperdicio en Manufactura (TPS)	Equivalente en Desarrollo de Software	Impacto Técnico y Ejemplos de Cátedra
6	Espera	**Demoras / Esperas (*Delays / Waiting*)**	Esperar respuestas del cliente, aprobaciones del CCB, entornos de testing o revisiones de código que detienen el avance de las tarjetas.
7	Defectos	**Defectos (*Bugs / Rework*)**	Errores que obligan a investigar, corregir, re-compilar y ejecutar pruebas de regresión; su costo crece exponencialmente cuanto más tarde se detectan.
8	*(Octavo desperdicio)*	**Talento Humano Desaprovechado**	Subutilizar el criterio e ingenio técnico del equipo limitándolo a ejecutar instrucciones mecánicas.

1.2 El Método Kanban en el Contexto del Desarrollo de Software (David J. Anderson)

- **¿Qué ES y qué NO ES Kanban? (Pregunta Clásica de Examen):** Kanban es un método evolutivo para definir, gestionar y mejorar servicios que entregan trabajo de conocimiento mediante un **sistema virtual de arrastre (*Pull*)** limitado por la capacidad (WIP). **NO es un ciclo de vida ni un proceso de desarrollo de software completo**, ni reemplaza al proceso existente, sino que se aplica sobre el proceso actual para visualizar el flujo y mejorarlo incrementalmente.
- **Principios de Gestión del Cambio:** (1) Empezar con lo que se hace ahora; (2) Acordar buscar la mejora mediante un cambio evolutivo; (3) Fomentar actos de liderazgo en todos los niveles.
- **Principios de Entrega de Servicio:** (1) Entender las necesidades y expectativas del cliente y enfocarse en ellas; (2) **Gestionar el trabajo y dejar que las personas se autoorganicen en torno a él**; (3) Evolucionar regularmente las políticas para mejorar los resultados.
- **Las 6 Prácticas Generales de Kanban:**
  1. **Visualizar el trabajo:** Mapear cada etapa real del flujo en columnas (divididas en subcolumnas *En Curso / Doing* y *Listo / Done / Buffer*) y representar cada ítem como una tarjeta señalizadora.
  2. **Limitar el Trabajo en Progreso (*Limit WIP*):** Fijar un tope numérico máximo de tarjetas por columna (abarca *En Curso + Listo*), creando un **sistema Pull** donde una etapa anterior solo puede avanzar un ítem cuando la etapa siguiente tiene cupo libre.
  3. **Gestionar el flujo (*Manage Flow*):** Maximizar el flujo continuo y predecible de valor, atacando bloqueos, colas y cuellos de botella (*"Stop starting, start finishing"*).
  4. **Hacer las políticas explícitas (*Make Policies Explicit*):** Definir reglas visibles y consensuadas sobre criterios de tracción (cuándo un ítem pasa a *Listo*), límites WIP, reposición y clases de servicio.
  5. **Implementar ciclos de retroalimentación (*Feedback Loops*):** Sostener las 7 cadencias (Standup diaria recorriendo el tablero de derecha a izquierda, Reaprovisionamiento, Planificación de Entrega, Revisión de Entrega de Servicio, Revisión de Operaciones, Revisión de Riesgos y Revisión de Estrategia).
  6. **Mejorar colaborativamente, evolucionar experimentalmente:** Aplicar modelos científicos (Ley de Little, Teoría de las Restricciones) para formular hipótesis de mejora y medir su efecto.
- **Los 5 Ejes de Políticas Explícitas de Calidad y Distribución de Capacidad de Cátedra:** Las políticas explícitas se redactan sobre: (1) **Construcción por desarrolladores** (estándares de código, pruebas unitarias), (2) **Testing de producto** (cobertura de pruebas de aceptación), (3) **Gestión de Configuración - SCM** (commits, ramas, pull requests), (4) **Gestión de Defectos** (priorización de bugs críticos) y (5)

Mejora Continua (*swarming* ante bloqueos y ajustes de WIP). La distribución típica de capacidad del equipo recomendada por la cátedra es: **60% Nuevas Funcionalidades (US/Casos de Uso)**, **20% a 30% Mantenimiento/Refactorización** y **10% a 20% Resolución de Bugs/Defectos**.

Clase de Servicio	Comportamiento del Costo de Demora	Política de Tratamiento en el Tablero y Ejemplo
1. Expedite (Urgente)	**Inmediato y altísimo** (el negocio pierde dinero o está frenado ahora mismo).	Prioridad máxima absoluta; se permite **máximo 1 Expedite a la vez** en el tablero; puede poner en pausa otras tareas. *Ej.: Caída de producción o falla crítica de cobros.*
2. Fecha Fija (*Fixed Date*)	**Escalón en fecha límite** (costo alto o multa si supera el vencimiento).	Ingresa al tablero calculando su *Lead Time* para llegar antes de la fecha; tiene prioridad sobre Estándar si se acerca el vencimiento. *Ej.: Normativa legal de AFIP vigente desde el 1/11.*
3. Estándar (*Standard*)	**Lineal y gradual** a lo largo del tiempo.	Se procesa por orden de llegada (**FIFO**) respetando estrictamente los límites WIP. *Ej.: Historias de usuario evolutivas habituales.*
4. Intangible	**Bajo hoy, exponencial a futuro** si nunca se atiende (deuda técnica).	Se le reserva un porcentaje de capacidad (20%-30%) para que no sea postergado eternamente. *Ej.: Refactorización arquitectónica o migración de librerías.*

1.3 Métricas en los 3 Enfoques: Tradicional (EVM), Ágil y Lean/Kanban

Conceptos Base: Una **Medida** es la cuantificación puntual de un atributo (ej. 20 defectos); una **Métrica** relaciona matemáticamente medidas (ej. defectos/KLOC); y un **Indicador** contrasta métricas contra un umbral o meta para **tomar decisiones de gestión**.

- **Métricas Tradicionales de Producto:** Orientadas al tamaño (**LOC / KLOC**: simples pero dependientes del lenguaje) y orientadas a la función (**Puntos de Función - FP**: independientes del lenguaje, calculados a partir de Entradas, Salidas, Consultas, Archivos Lógicos Internos e Interfaces Externas), además de la **Eficiencia de Remoción de Defectos** (DRE = E / (E + D), donde E son errores detectados antes de la entrega y D defectos reportados en producción).
- **Métricas Tradicionales de Proyecto — Gestión del Valor Ganado (EVM / *Earned Value Management*):**

Métrica EVM (Inglés / Español)	Fórmula Matemática	Significado e Interpretación en Examen
BAC (CPF)	Σ PV total del proyecto	Presupuesto total original aprobado al finalizar el proyecto.
PV (CPTP)	BAC × % Avance Planificado	Valor Planificado: presupuesto que *debería* haberse ejecutado según el cronograma hasta la fecha de corte.
EV (CPTR)	BAC × % Avance Real Terminado	**Valor Ganado:** cuánto vale (en moneda presupuestada) el trabajo que **realmente se completó** hasta hoy.
AC (CRTR)	Σ Costos reales devengados hoy	Costo Real: dinero efectivamente gastado hasta la fecha para lograr el trabajo realizado.

Métrica EVM (Inglés / Español)	Fórmula Matemática	Significado e Interpretación en Examen
SV (VC) y **SPI (IPC)**	SV = EV − PV SPI = EV / PV	**Desempeño del Cronograma:** Si SV < 0 y SPI < 1, el proyecto está **atrasado** respecto al plan. Si SV > 0 y SPI > 1, está adelantado.
CV (VCosto) y **CPI (IPCosto)**	CV = EV − AC CPI = EV / AC	**Desempeño del Costo:** Si CV < 0 y CPI < 1, el proyecto tiene **sobrecosto** (gasta más de lo que rinde). Si CV > 0 y CPI > 1, ahorra presupuesto.
EAC (CEF) y **ETC (ETF)**	EAC = BAC / CPI · ETC = EAC − AC	**Proyección Final: EAC** estima cuánto terminará costando todo el proyecto si persiste la eficiencia actual; **ETC** indica cuánto dinero falta gastar desde hoy hasta terminar.

- **Métricas en el Enfoque Ágil (Scrum):** (1) **Velocidad (*Velocity*):** Suma de Story Points de las historias que cumplen íntegramente la DoD al cierre del Sprint (métrica de progreso y predicción interna, nunca comparable entre equipos); (2) **Sprint Burndown Chart:** Trabajo restante vs. tiempo del Sprint; (3) **Release / Sprint Burnup Chart:** Muestra en dos curvas separadas el trabajo completado acumulado y la línea de alcance total, permitiendo distinguir si un retraso se debe a menor ritmo del equipo o a un aumento de alcance (*scope creep*).
- **Métricas en el Enfoque Lean / Kanban y Ley de Little:**
  - **Lead Time:** Tiempo total desde que el cliente realiza el pedido hasta que se le entrega en producción.
  - **Cycle Time (Tiempo en Proceso - TiP):** Tiempo desde que el equipo comienza a trabajar en la tarjeta hasta que la termina.
  - **Touch Time y Eficiencia de Flujo:** Tiempo real de trabajo activo sin esperas (Eficiencia = Touch Time / Lead Time).
  - **Throughput (Tasa de Entrega):** Cantidad de ítems completados por unidad de tiempo (ej. tarjetas/semana).
  - **Ley de Little:** Cycle Time Promedio = WIP Promedio / Throughput Promedio. Demuestra matemáticamente que **para reducir el tiempo de entrega (Cycle/Lead Time) es indispensable limitar y reducir el WIP**.
  - **Diagrama de Flujo Acumulado (CFD):** Gráfico de áreas apiladas por etapa donde la distancia vertical entre curvas mide el **WIP**, la distancia horizontal mide el **Lead/Cycle Time promedio** y la pendiente de la curva inferior mide el **Throughput**. Si una franja se ensancha verticalmente en el tiempo, revela un **cuello de botella** en esa etapa.
- **Design Thinking y Retrospectivas (Derby & Larsen):** Design Thinking estructura el descubrimiento en 5 pasos (*Empatizar, Definir, Idear, Prototipar y Probar*). Por su parte, las **Retrospectivas** inspeccionan el proceso y las relaciones del equipo a través de **5 Etapas**: (1) *Preparar el escenario* (Directiva Primaria y seguridad psicológica), (2) *Recolectar datos* (hechos y métricas de la iteración), (3) *Generar insights* (analizar causas raíz con 5 Porqués o Ishikawa), (4) *Decidir qué hacer* (definir 1 a 3 acciones SMART concretas para el próximo ciclo) y (5) *Cerrar la retrospectiva*.

2. Unidad 03 (Parte II): Prácticas Continuas y Estrategias de Despliegue

Fuentes de cátedra (Notebook ISW): `Compiladao parcial segundo parcial`, `02 SCM.pdf`, `RESUMEN FINAL -ISW- ALEX.pdf`.

2.1 Prácticas Continuas: CI vs. Continuous Delivery (CDE) vs. Continuous Deployment (CD)

- **Integración Continua (*Continuous Integration - CI*):** Práctica en la que los desarrolladores fusionan sus cambios en la rama principal compartida (*trunk/main*) **varias veces al día**; cada commit dispara automáticamente un servidor de CI que compila el código y ejecuta pruebas unitarias y de integración para detectar roturas de inmediato.
- **Entrega Continua (*Continuous Delivery - CDE*):** Extiende la CI automatizando todo el pipeline de pruebas de sistema/aceptación y empaquetado para garantizar que el software esté **siempre en estado listo para desplegarse en producción**; sin embargo, el pase final a producción se activa mediante una **decisión humana manual ("un solo click")**. (La CI es prerrequisito obligatorio de la CDE).
- **Despliegue Continuo (*Continuous Deployment - CD*):** Extiende la Entrega Continua eliminando el paso manual: **todo cambio que aprueba exitosamente las pruebas automatizadas del pipeline se despliega automáticamente en producción para los usuarios finales sin intervención humana**.

2.2 Comparativa de Estrategias de Despliegue de Producto

Estrategia	Mecanismo Operativo	Downtime	Rollback, Costo y Caso de Uso Ideal
Big Bang / Recreate	Se apaga toda la versión actual (v_1) y se reemplaza por completo con la nueva versión (v_2).	**Alto** (corte total del servicio).	Rollback lento y riesgoso; impacto del 100% ante fallas. Solo para sistemas internos no críticos con ventana nocturna.
Rolling Update (Gradual)	Actualiza los nodos o instancias del clúster de a uno por vez de v_1 a v_2.	**Cero**.	No duplica hardware; exige compatibilidad simultánea de v_1 y v_2 con el esquema de base de datos.
Blue-Green Deployment	Mantiene 2 entornos idénticos: *Blue* (v_1 activo) y *Green* (v_2 probado en paralelo); el balanceador conmuta todo el tráfico a *Green*.	**Cero**.	**Rollback instantáneo** (volver el router a *Blue*); desventaja: **alto costo de infraestructura** al duplicar producción.
Canary Release (Canario)	Libera v_2 inicialmente solo a un **1%-5% de los usuarios**; si las métricas y errores son saludables, se incrementa gradualmente al 100%.	**Cero**.	**Riesgo mínimo en producción real** (si falla, solo afecta al 5% y se revierte al instante) sin duplicar toda la infraestructura.
Feature Toggles / Flags y **Shadow**	*Toggles:* despliega el código apagado bajo una bandera condicional activable sin redesplegar. *Shadow:* duplica tráfico real hacia v_2 en segundo plano sin mostrar su salida al cliente.	**Cero**.	Permiten desacoplar el *deploy técnico* del *release comercial* y probar rendimiento real sin impacto en el usuario.

3. Unidad 04: Aseguramiento de Calidad de Proceso y de Producto

Fuentes de cátedra (Notebook ISW): `12 PPQA_Intro.pdf`, `13 Revisiones Técnicas.pdf`, `14 Auditorías de Sw.pdf`, `09 Testing de Software.pdf`, `09 Testing de Software Estrategias Caja Negra.pdf`, `09 Agile Testing.pdf`, `TDD.pdf`.

3.1 Conceptos Generales de Calidad y Costos de la Calidad (CoQ)

- **Definiciones de Calidad según Autores Clásicos:**
  - **Philip Crosby:** *"Conformidad con los requerimientos"* (sistema basado en la prevención y estándar "Cero Defectos").
  - **Joseph Juran:** *"Adecuación al uso"* (*Fitness for use*) según las necesidades reales del cliente.
  - **W. Edwards Deming:** Reducción de la variabilidad del proceso estadístico y mejora continua mediante el ciclo **PDCA (Plan-Do-Check-Act)**.
  - **ISO 9000 / Pressman:** Grado en que un conjunto de características inherentes cumple con los requisitos funcionales y de rendimiento explícitos, los estándares de desarrollo documentados y las características implícitas esperadas de todo software profesional.
- **Calidad de Diseño vs. Calidad de Concordancia:** La **Calidad de Diseño** mide qué tan bien la especificación y arquitectura proyectadas responden a las necesidades del usuario; la **Calidad de Concordancia** mide qué tan fielmente la implementación construida respeta ese diseño y los estándares establecidos.
- **Clasificación de los Costos de la Calidad (CoQ):**
  1. **Costos de Conformidad (Inversión controlada):** Se dividen en **Costos de Prevención** (planificación de calidad, capacitación, definición de procesos y estándares) y **Costos de Evaluación / Inspección** (revisiones técnicas, auditorías, diseño y ejecución de pruebas).
  2. **Costos de No Conformidad (Pérdidas por defectos):** Se dividen en **Costos de Fallas Internas** (detectados *antes* de entregar al cliente: debugging, retrabajo, re-testing) y **Costos de Fallas Externas** (detectados en producción por el usuario: soporte, parches de urgencia, penalidades legales, pérdida de reputación). *Detectar un defecto en requerimientos o en una revisión técnica cuesta entre 10 y 100 veces menos que repararlo en producción.*

3.2 Calidad de Proceso, Modelos CMMI / ISO / SPICE y el Área PPQA

Premisa central: La calidad del producto está determinada por la calidad del proceso usado para construirlo. El modelo **CMMI-DEV** ofrece dos representaciones: (1) **Continua** (evalúa áreas individuales por **Niveles de Capacidad 0 a 3**: 0 Incompleto, 1 Realizado, 2 Gestionado, 3 Definido) y (2) **Escalonada (Por Etapas)** (evalúa a toda la organización por **5 Niveles de Madurez**: **1 Inicial**, **2 Gestionado** —con 7 áreas clave: REQM, PP, PMC, SAM, MA, CM y PPQA—, **3 Definido** a nivel organizacional, **4 Gestionado Cuantitativamente** y **5 En Optimización**).

Meta Específica de PPQA (Nivel 2 Soporte)	Prácticas Específicas (SP)	Aplicación y Regla de Escalamiento en CMMI
SG 1: Evaluar objetivamente los procesos y productos de trabajo	• **SP 1.1:** Evaluar objetivamente los procesos. • **SP 1.2:** Evaluar objetivamente los productos de trabajo y servicios.	El grupo de calidad contrasta de forma independiente las actividades ejecutadas y los artefactos producidos contra los planes, estándares y procedimientos definidos.
SG 2: Proporcionar una visión objetiva (Comunicar y resolver no conformidades)	• **SP 2.1:** Comunicar y asegurar la resolución de las cuestiones de no conformidad. • **SP 2.2:** Establecer y mantener registros de las actividades de QA.	Las desviaciones (no conformidades) se tratan de resolver primero dentro del proyecto con los responsables; **si no se pueden resolver en el ámbito del proyecto, se escalan a la gerencia superior**.

Estándares ISO complementarios: ISO 9001 (Gestión de calidad certificable genérica), **ISO/IEC 12207** (Procesos del ciclo de vida del software), **ISO/IEC 15504 - SPICE** (Evaluación bidimensional de capacidad

de procesos 0 a 5) y la familia **ISO/IEC 25000 - SQuaRE** (donde **ISO 25010** define las 8 características de calidad de producto: *Adecuación funcional, Eficiencia de desempeño, Compatibilidad, Usabilidad, Fiabilidad, Seguridad, Mantenibilidad y Portabilidad* + 5 de calidad en uso; e **ISO 25012** define la calidad de datos).

3.3 Revisiones Técnicas del Software vs. Auditorías de Software

Criterio Comparativo	Revisiones Técnicas (Ej. Inspección Formal de Fagan, Walkthrough, Pares)	Auditorías de Software (Auditorías de Proyecto, al Grupo de Calidad y de Configuración)
Quién la ejecuta y sobre qué	**Pares técnicos del proyecto** revisan estáticamente un **producto de trabajo** (ERS, diseño, código, casos de prueba) sin ejecutarlo.	**Auditores independientes** (SQA/PPQA o externos) evalúan el **proceso o configuración** mediante muestreo de **evidencia objetiva**.
Propósito principal	Descubrir **errores y defectos técnicos** temprano para que el autor los corrija en el retrabajo (evalúa al producto, nunca al autor).	Verificar la **conformidad (cumplimiento)** respecto a estándares, normas y planes, detectando **no conformidades (desviaciones)**.
Roles, Etapas y Parámetros Clave	**Inspección de Fagan:** 5 roles (*Moderador* —líder—, *Autor*, *Lector*, *Inspectores*, *Escriba*); 6/7 etapas (*Planificación, Vista Previa, Preparación Individual con checklists, Reunión de Inspección ≤ 2 hs sin resolver bugs en sala, Retrabajo y Seguimiento*). Tasa: 100-250 LOC/h o 3-5 págs/h (3 a 5 personas). *Walkthrough:* Semiformal, la dirige el propio Autor sin preparación obligatoria previa.	**Tipos:** (1) *Auditoría de Proyecto* (SQA audita al proyecto); (2) *Auditoría al Grupo de Calidad* (auditor externo audita a SQA); (3) *Auditoría de Configuración* (FCA funcional y PCA física). **Etapas:** Planificación → Apertura → Recolección de Evidencias → Informe de Hallazgos/No Conformidades → Seguimiento de Acciones Correctivas. **Métricas:** Esfuerzo, cantidad de desviaciones y duración.

3.4 Calidad de Producto y Testing de Software (Tradicional y Ágil)

- **Cadena Causal Error → Defecto → Falla:** El **Error** es la equivocación humana del desarrollador/analista; el **Defecto (Bug / Fault)** es la imperfección física plasmada en el artefacto o código (nota de cátedra: si la imperfección es detectada y corregida internamente dentro de la misma etapa antes de entregar el artefacto se denomina *error*, y si escapa de la fase hacia pruebas/producción se contabiliza como *defecto*); la **Falla (Failure)** ocurre cuando el código defectuoso se ejecuta y produce un comportamiento externo incorrecto.
- **Severidad vs. Prioridad:** La **Severidad** mide la gravedad del impacto técnico/funcional en el sistema; la **Prioridad** mide la urgencia de negocio para repararlo.
- **Verificación vs. Validación (Modelo V) y Principios: Verificación** comprueba si *"estamos construyendo el producto correctamente"* (conformidad con la especificación); **Validación** comprueba si *"estamos construyendo el producto correcto"* (satisface la necesidad real del usuario). El testing **demuestra la presencia de defectos, nunca su ausencia**; como las pruebas exhaustivas al 100% son imposibles, el testing mitiga el riesgo pero no garantiza calidad por sí solo.
- **Los 4 Niveles de Prueba y Estrategias de Integración:**
  1. **Pruebas Unitarias:** Prueban cada método/clase de forma aislada (caja blanca / TDD).
  2. **Pruebas de Integración:** Verifican la interacción entre módulos. Estrategias incrementales: (a) **Top-Down (Descendente):** inicia desde el módulo de control principal hacia abajo reemplazando los submódulos inferiores aún no construidos por **Stubs (Sustitutos)**; (b) **Bottom-Up (Ascendente):** inicia

probando los módulos de nivel más bajo invocándolos mediante **Drivers (Controladores/Conductores)**; (c) **Sandwich:** híbrida.

3. **Pruebas de Sistema:** Evalúan el sistema completamente integrado en sus requisitos funcionales y no funcionales (rendimiento, carga, estrés, seguridad), incluyendo **Pruebas de Humo (*Smoke Testing*)** y **Pruebas de Regresión** (re-ejecutar pruebas tras un cambio para asegurar que no se introdujeron efectos colaterales en lo que ya funcionaba; en el *Ciclo 0* se ejecutan todos los casos y en los ciclos siguientes se re-testean los fallidos más la regresión).
4. **Pruebas de Aceptación:** Validación con el cliente: **Pruebas Alfa (α)** (por usuarios en el entorno controlado del desarrollador), **Pruebas Beta (β)** (por usuarios finales en su propio entorno real sin presencia del desarrollador) y **UAT**.

- **Técnicas de Caja Blanca y Complejidad Ciclomática de McCabe V(G):** Buscan cubrir sentencias, ramas (`True`/`False`) y condiciones lógicas. La **Complejidad Ciclomática V(G)** determina el número de caminos básicos linealmente independientes (cota superior de casos de prueba para cubrir el 100% de las ramas) mediante 3 fórmulas equivalentes: **Fórmulas de McCabe:** V(G) = Regiones │ V(G) = Aristas − Nodos + 2 │ V(G) = Nodos Predicado (P) + 1
- **Testing en Ambientes Ágiles, Pirámide de Mike Cohn, TDD y BDD/ATDD:**
  - **Los 4 Cuadrantes de Agile Testing (Crispin & Gregory): Q1** (Tecnología + Apoya al equipo: *Pruebas Unitarias y de Componentes*, 100% automatizadas); **Q2** (Negocio + Apoya al equipo: *Pruebas Funcionales, Ejemplos, Historias de Usuario, Prototipos*, automatizadas/manuales); **Q3** (Negocio + Critica al producto: *Testing Exploratorio, Usabilidad, Alfa/Beta, UAT*, 100% manuales); **Q4** (Tecnología + Critica al producto: *Rendimiento, Carga, Estrés, Seguridad*, automatizadas con herramientas).
  - **Pirámide de Automatización de Cohn:** Base ancha de **Pruebas Unitarias** (rápidas, baratas y estables, 60%-70%), capa media de **Pruebas de Servicios/API** (20%-30%) y cúspide pequeña de **Pruebas de UI** (lentas y frágiles, 5%-10%).
  - **TDD (*Test-Driven Development* - Kent Beck):** Técnica de diseño guiada por pruebas con el ciclo **Red (escribir test que falla) → Green (escribir el código mínimo para que pase) → Refactor (limpiar el diseño manteniendo los tests en verde)**. Patrones Green Bar de Beck: *Fake It* (devolver constante primero), *Triangulación* (generalizar con 2+ ejemplos) y *Obvious Implementation*.
  - **ATDD y BDD:** En ATDD los "Tres Amigos" (PO + Dev + Tester) definen pruebas de aceptación antes de programar; BDD expresa esos escenarios con lenguaje Gherkin (**Dado / *Given* – Cuando / *When* – Entonces / *Then***).

4. Guía Práctica de Resolución del Segundo Parcial (Caja Negra y Kanban)

Fuentes de cátedra (Notebook ISW): `09 Testing de Software Estrategias Caja Negra.pdf`, `Template Caso de Prueba.docx`, `ejercicio clase testing (1).pdf`, `parcial2-2026-4k4.pdf`, `parcial2-2025-4k1.pdf`.

4.1 Algoritmo para el Ejercicio Práctico de Diseño de Casos de Prueba (Partición de Equivalencia y Valores Límite)

1. **Paso 1 — Derivar las Clases de Equivalencia Válidas (CEV) y No Válidas (CENV) + Valores Límite (VL):**
   - Si una entrada es un **rango [a, b]** (ej. *Edad entre 18 y 65*): definir **1 CEV** (18 ≤ E ≤ 65) y **2 CENV** (E < 18 y E > 65), más los **Valores Límite exactos en las fronteras** (a−1, a, b, b+1 → 17, 18, 65, 66).
   - Si una entrada es un **conjunto discreto de opciones** (ej. *Rubro: Joyería, Pintura, Escultura*): definir las CEV para las opciones admitidas y **1 CENV** para un valor fuera del conjunto o vacío.
   - Si una entrada es una **condición lógica o formato único**: definir **1 CEV** (cumple la condición) y **1 CENV** (no la cumple).

2. **Paso 2 — Aplicar las 2 Reglas de Oro de Combinación en los Casos de Prueba (Criterio Estricto de Cátedra):**
   - **Regla para Clases Válidas (Camino Feliz): Agrupar/combinar la mayor cantidad posible de Clases de Equivalencia Válidas en un mismo Caso de Prueba** hasta cubrir todas las CEV de todas las entradas con el menor número de casos válidos.
   - **Regla para Clases No Válidas (Prevención del Enmascaramiento de Defectos / *Fault Masking*): Diseñar UN Caso de Prueba separado por cada Clase de Equivalencia No Válida individual, manteniendo TODOS los demás campos de entrada con valores VÁLIDOS.** Nunca mezclar dos entradas inválidas en el mismo caso de prueba, ya que el primer error detendría la validación e impediría verificar si el segundo error es detectado por el sistema.
3. **Paso 3 — Completar todos los Campos del Template Oficial de Caso de Prueba de la Cátedra:**

Campo del Template	Cómo redactarlo correctamente en el Segundo Parcial
ID y Nombre del Caso	Identificador único y objetivo claro (ej. `CP-01: Solicitud de préstamo válida en límite inferior de monto y cuotas`).
Precondiciones	Estado previo necesario del sistema y datos maestros (ej. *"Cliente registrado con cuenta activa y logueado en la pantalla 'Solicitar Préstamo'"*).
Datos de Entrada (¡Valores Concretos!)	Escribir **valores literales exactos** para cada variable (ej. `Edad = 18`, `Monto = $100.000`, `Moneda = ARS`) indicando qué CEV/CENV y VL cubren. **Jamás poner rangos abstractos como "Monto > 0" en los datos de entrada del caso.**
Pasos de Ejecución	Acciones numeradas concretas (1. Completar los campos con los datos de entrada; 2. Presionar el botón *"Enviar Solicitud"*).
Resultado Esperado	Respuesta exacta e inequívoca del sistema (ej. en válido: *"Se aprueba la solicitud y se muestra comprobante"*; en inválido: *"El sistema rechaza el registro e informa 'La edad mínima requerida es de 18 años'"*).

4.2 Algoritmo para Diagnosticar y Resolver el Ejercicio de Tablero Kanban

1. **Identificar el Cuello de Botella:** Sumar las tarjetas presentes en las subcolumnas *En Curso* + *Listo* de cada etapa y compararlas con su límite WIP; señalar qué columna alcanzó su límite WIP (bloqueando por efecto *Pull* la salida de las etapas anteriores) y qué columnas posteriores sufren inanición (*starvation*).
2. **Proponer Soluciones Alineadas con el Método Kanban:** (a) Respetar el límite WIP sin "empujar" (*push*) más tarjetas a la etapa saturada; (b) Aplicar colaboración grupal (**Swarming**) para que miembros de otras etapas ayuden a destrabar las tareas acumuladas en el cuello de botella; (c) Reasignar capacidad o ajustar los límites WIP de las columnas previas para equilibrar el flujo; y (d) Redactar **Políticas Explícitas de Calidad** (sobre los 5 ejes de la cátedra) que reduzcan los defectos y retrabajos que originaron el tapón.